

The Polyglot's Shorthand

An 18M-Parameter Encoder for Latinized Code-Mixed Text

Inter IIT Bootcamp 2026 · CS/AI Practice Problem Statement 4

2026-09-19 · @Ashish K

Abstract

Consumer text across South Asia arrives Latinised and code-mixed — *bhai order cancel krdo please, urgent meeting h*. Native-script Indic encoders fragment it, English encoders read the syntax as noise, and frontier LLMs handle it well but too slowly to sit at the front of a routing queue.

We built a small encoder for this text specifically: a deterministic phonetic normalisation layer, a byte-level BPE vocabulary trained over canonicalised forms, and a transformer whose input embedding carries four channels — subword identity, position, a hashed consonant skeleton, and six surface features that canonicalisation throws away. The skeleton channel is the main idea: `krdo` and `kardo` tokenise to `kr|do` and `kar|do`, sharing no subword, but both reduce to the skeleton `krd` and so share an embedding.

The model is **18,868,240 parameters — 3.8% of the 500M budget** — trained from scratch on 1.81M tokens. On our split it reaches **0.872 weighted F1** on SentiMix Hinglish sentiment and **0.970** on HingLID token-level language identification, retains **88.5%** of sentiment F1 under heavy spelling perturbation, and serves at **3.02 ms end-to-end at batch 1** with a peak of **12,742 sentences/s**. Stripping emoji costs **6.9 F1 points** on the 232 test rows that carry one.

Two caveats bind the headline number and are developed in §9: our test set is half the labelled validation file rather than the official held-out set, and the encoder's MLM stage saw the unlabelled text of that split. The number is not a leaderboard result and we do not present it as one.

1 · Why the existing options fail

The traffic is the ingestion queue of a South Asian consumer platform — support chats, reviews, delivery queries. Three approaches fail on it, each differently. Native-script encoders like IndicBERT and MuRIL carry Devanagari vocabularies, so a Romanised Hindi word has no merge path and breaks into character fragments or falls to `[UNK]`; transliterating back first is lossy in exactly the ambiguous cases, since `kr` could be *kar*, *kir* or *kur* and the transliterator must guess before the encoder sees anything. English

encoders handle the embedded English words fine but find the Hindi matrix syntax holding them together — verb-final order, postpositions, auxiliaries — unmotivated. Frontier LLMs fail economically rather than linguistically: a router classifying every message in single-digit milliseconds cannot call a large model, however accurate.

The problem statement names three properties to absorb, each mapping to something concrete here: phonetic volatility (canonicalisation and the skeleton channel), intra-sentence mixing (one shared vocabulary, with token-level language ID as an auxiliary task), and pragmatic flips via emoji (atomic emoji tokens and the surface-feature channel). The third is easiest to lose — stripping emoji as noise deletes the token carrying the sentiment, and 17.4% of our training tweets contain at least one.

The argument is that the gap between *parses this text* and *deployable at the front of a queue* is closed by the input representation, not by a smaller general model. If the representation absorbs spelling variation before the transformer sees it, the transformer can be small.

2 • Data

Real corpora only; the perturbation operators in §3 augment real sentences and stress-test held-out ones, and nothing was manufactured. We use SemEval-2020 Task 9 (SentiMix Hinglish) for sentence sentiment — 15,130 train, 3,000 labelled validation, 3,000 unlabelled, distributed 5,637 neutral / 5,034 positive / 4,459 negative — and L3Cube-HingLID for word-level Hindi/English tags, 31,756 / 6,279 / 6,420 sentences running 693,977 HI to 274,255 EN. SentiMix also carries its own `Hin / Eng` word tags, which we map onto the HingLID label space and fold into the language-ID task: a further 170,733 HI and 90,551 EN tokens, useful because it puts code-switching supervision inside the same domain as the sentiment data.

Evaluation protocol. The SemEval gold test labels were never released. We train on the 15,130-row train file and split the labelled validation file in half: 1,500 for model selection, 1,500 held out. Model selection uses validation weighted F1 on sentiment alone — the language-ID head can improve while sentiment degrades.

Two data problems. The labelled files are clean UTF-8, but the unlabelled `Test.txt` was written UTF-8-as-cp1252, so every emoji arrives as `ðŸ~...`. Round-tripping through cp1252 repairs 2,431 of 2,642 damaged tokens and recovers 610 emoji; our first attempt used latin-1 and fixed 24. Separately, SentiMix splits `@handle` into two tokens and URLs into fragments — about 21,900 and 8,500 tokens of noise in train. We collapse both to `<user>` and `<url>`, collapsing the tag sequence in lockstep, and assert alignment afterwards; a silent desync there would train the language-ID head on offset labels and still look plausible.

The pretraining corpus is every sentence from both sources, labelled or not: **65,585 sentences, 1,813,240 tokens**, 27.6 tokens per sentence. That is small, and §9 treats it as the main limitation.

3 · Tokenisation

The pipeline is word-centric: each word becomes three representations independently and the subword sequence is assembled afterwards, since a word-level property can't be attached to a stream that has forgotten where words began. Segmentation is NFKC plus splitting emoji runs and punctuation into their own tokens. Canonicalisation lowercases, caps character runs at two, applies 18 digraph folds (`aa→a` , `ee→i` , `ph→f` , `sh→s` , `ai→e` , ...) and a single-character map (`z→j` , `w→v` , `q→k` , `y→i`), then collapses what the folding produced — the aim isn't correct phonology, it's making independent writers of a word converge on one string.

The vocabulary is byte-level BPE, 16,000 merges plus 1,291 emoji and placeholder tokens added atomically afterwards, for a final size of **17,291**. Byte-level is the choice that matters: there is no out-of-vocabulary path, so an unfamiliar spelling produces more subwords rather than `[UNK]`. Sequence length is the cheap axis to degrade on; meaning is the expensive one.

Measured fertility is **1.369 subwords per word** on clean text and **1.477** after two rounds of perturbation — a 7.9% increase. That 7.9% is the tokeniser's own contribution to drift robustness, before the model contributes anything: heavy misspelling costs less than one extra subword per twelve words. The surface-space collapse across the corpus makes the same point from the other direction — 108,637 raw surface words become 72,871 canonical forms (−32.9%) and 27,182 consonant skeletons (−75.0%). A third of this corpus's apparent vocabulary is spelling variation; by the aggressive measure, three-quarters of it is.

The subword probes also show why a second channel is needed. `krdo` → `kr|do` and `kardo` → `kar|do` share no subword at all; canonicalisation alone does not merge them. Folding is imperfect elsewhere too — `bohoooot` → `bo|h|ut` , `chahiye` → `c|ahie` — so the vocabulary is not doing this job by itself.

Drift is modelled by ten operators — vowel doubling and substitution, consonant substitution (`k→c` , `v→w` , `j→z` , `f→ph`), vowel deletion (`karna` → `krna`), terminal addition or deletion, consonant doubling, adjacent transposition — applied up to n times per word at probability 0.55 each for strength n .

4 · The model

The input embedding sums four terms into one 384-dimensional space: subword identity over canonicalised forms, learned absolute position, a hashed consonant skeleton, and a projection of six surface features. Skeleton ids and features are computed per word and replicated across that word's subwords.

The skeleton channel. A word's canonical form reduced to first character plus non-initial consonants, truncated at 8, hashed with CRC32 into 8,192 buckets. `kar`, `kr`, `karr` and `kaar` all reach `kr`; `nahi`, `nhi` and `nahee` reach `nh`; `cancel`, `cancell` and `canc1` reach `cnc1`; and the `krdo` / `kardo` pair from §3, which shares no subword, reaches `krd` from both sides. A skeleton under two characters falls back to the canonical form, so `ki`, `ka` and `ko` stay distinct instead of collapsing onto `k`. Collisions are frequent — 27,182 skeletons into 8,191 buckets, about 3.3 each — tolerable because the subword channel disambiguates, but `skeleton_buckets` is a real hyperparameter we didn't tune.

The surface-feature channel restores six values canonicalisation destroys: is-emoji, is-punctuation, contains-digit, elongation, all-caps, length ÷ 12. Elongation and caps are the intensity markers this register uses — *bohoooot* is not *bohot*.

Encoder. Six pre-LN blocks, `d_model` 384, 6 heads, FFN 1,152, GELU, dropout 0.1. Pre-LN because it trains stably from random init without a tuned warmup, and we have no checkpoint to fall back on. Attention goes through `F.scaled_dot_product_attention` so PyTorch selects the fused kernel. The mask is boolean rather than an additive `-inf` float, avoiding a dtype mismatch under fp16 autocast. Four heads share the encoder: MLM weight-tied to the input embedding, sentiment over pooled `[CLS]`, a per-token language-ID tagger labelling the first subword of each word, and a serving graph of encoder plus sentiment head.

Component	Parameters
Token embedding (17,291 × 384)	6,639,744
Skeleton embedding (8,192 × 384)	3,145,728
6 × transformer block	8,875,008
Position, features, norms, pooler, heads	207,760
Total	18,868,240
Against the 500M ceiling	3.77%

At 75.5 MB in fp32 and 18.9 MB in int8, more than half the parameters are lookup tables rather than compute, so shrinking this model means shrinking the vocabulary or bucket count, not removing layers. The $26\times$ headroom under the ceiling means `d_model=768` with 12 layers is comfortably affordable — we are limited by pretraining data, not the parameter budget.

5 • Training

Pretraining. Masked language modelling over all 65,585 sentences from random initialisation, 15% of non-special positions split 80/10/10 into `[MASK]`, a random token, and unchanged. AdamW at $3e-4$, OneCycle with 10% warmup, gradient clipping at 1.0, fp16 autocast, 14 epochs in **19.7 minutes** on a T4. Loss falls $7.41 \rightarrow 3.12$ (perplexity $1644 \rightarrow 22.7$), still improving slowly at the end — the curve suggests more epochs would help marginally and more data would help a lot.

The skeleton channel is zeroed at masked positions. Without that the task is nearly degenerate, since the skeleton of the masked word would still be in the input and loss would fall faster while the model learned less.

Defect in this run. `skel_ids` is masked but `feats` is not, so at a masked position the model still sees that word's length, elongation and is-emoji flags. It makes pretraining slightly easier than intended. Fine-tuning and evaluation are unaffected; the fix is one line.

Fine-tuning. One encoder, two heads, batches interleaved and shuffled each epoch. Sentiment on SentiMix train (236 batches/epoch) at weight 1.0; language ID on HingLID plus SentiMix word tags (732 batches/epoch) at 0.5. AdamW at $1e-4$, batch 64, 6 epochs, label smoothing 0.05 — **4.9 minutes**. Validation sentiment F1 climbs $0.607 \rightarrow 0.869$ and is flat between epochs 4 and 5, so 6 epochs is the right stopping point. Despite language ID getting $3\times$ the gradient updates, sentiment did not appear starved.

Augmentation. 30% of training rows are re-spelled each epoch at strength 1 — real sentences, real labels, only the spelling moves. The robustness result below is therefore a joint claim about the architecture and the augmentation until the ablation in §10 separates them. Single seed (42) throughout.

6 • Results

Task	Accuracy	Macro F1	Weighted F1
Sentiment (n = 1,500)	0.8720	0.8735	0.8718

Task	Accuracy	Macro F1	Weighted F1
Language ID (n = 193,540 tokens)	0.9697	0.9633	0.9696

Sentiment is balanced across classes: negative 0.889 F1, neutral 0.855, positive 0.876. Language ID is 0.979 on Hindi and 0.948 on English, the gap tracking the 2.4:1 class imbalance.

On the 75.0 leaderboard figure. The best SemEval-2020 Task 9 system reported 0.750 weighted F1. Ours is 0.872 on our own split, and that comparison is not valid — we state this rather than claim a result. Three reasons. Our test set is half the labelled validation file, not the official held-out test set, and validation data in this shared task is drawn closer to the training distribution. The MLM stage consumed the unlabelled text of the validation and test halves, so the encoder is transductive with respect to its own test set — no label leakage, but not a clean inductive setup either. And 1,500 rows is a fifth of the official evaluation size. The defensible claim is narrower: an 18.9M-parameter encoder trained from scratch on 1.8M tokens is competitive in-domain with models two orders of magnitude larger, and the ablations in §10 are what would turn that into a measurement.

Robustness. The same 1,500 test sentences re-spelled at four strengths. Labels are invariant by construction, so any drop is brittleness.

Perturbation	Sentiment wF1	Language ID wF1	Sentiment retention
0 (control)	0.8718	0.9696	100%
1	0.8184	0.9494	93.9%
2	0.7758	0.9370	89.0%
3	0.7718	0.9246	88.5%

The curve flattens: levels 2 and 3 cost 0.4 F1 points between them against 5.3 points for level 1. Once spelling has drifted past the point where canonicalisation folds it, the skeleton channel is carrying the word and further corruption changes little. Language ID degrades more gently still, losing 4.5 points across the whole sweep, which is expected since it can fall back on subword shape.

Emoji. Stripping every emoji moves the full test set from 0.8718 to 0.8610 — small, because 84.5% of rows are unaffected. On the 232 rows that carry one, F1 falls **0.8783 → 0.8095, a 6.9-point drop**, while the 1,268 rows without emoji sit at 0.8704. The model is using the pragmatic channel, and at $n = 232$ a 6.9-point swing is well outside what sampling noise would produce. This is the cleanest confirmation in the study that emoji are functioning as semantic modulators rather than decoration.

7 · Error analysis

Code-mix ratio is the fraction of language-tagged tokens marked English; the median test row is 0.279 English.

Slice	n	Accuracy	Weighted F1
Mostly Hindi (EN < 20%)	554	0.8646	0.8628
Balanced (EN 20–60%)	622	0.8633	0.8633
Mostly English (EN > 60%)	324	0.9012	0.8998
Short (< 15 words)	178	0.8708	0.8717
Medium (15–30 words)	1,282	0.8690	0.8687
Long (> 30 words)	40	0.9750	0.9748
Contains emoji	232	0.8793	0.8783
Contains elongation	282	0.8936	0.8931
Low fertility (< 1.5)	1,210	0.8702	0.8701
All	1,500	0.8720	0.8718

The code-mix slices are the diagnostic that matters, and they come out well: mostly-Hindi 0.863, balanced 0.863, mostly-English 0.900. A 3.7-point edge on English-dominant rows is small enough to rule out the failure this design exists to avoid — the model is not merely reading the embedded English lexicon and guessing. Performance is flat across length and fertility, so fragmentation is not costing accuracy at inference. The high-fertility slice (≥ 2.0 subwords per word) had fewer than 20 rows and was dropped, which is itself informative: at 1.369 mean fertility, badly fragmented sentences are rare.

gold ↓ / predicted →	negative	neutral	positive
negative	393	28	9
neutral	43	487	48
positive	18	46	428

192 errors total. Neutral absorbs the most (91 of its 578 rows misread) and direct positive↔negative confusion is rare — 27 cases out of 922 polar rows, 2.9%. The model is reading stance, not just topic.

The eight most confident mistakes fall into one dominant pattern. Five are gold-neutral predicted positive on congratulatory or festival text: *"Many more days and years of good health good life and prosperity Happy Birthday"*, *"# EidMubarak"*, *"Thank you <user> for such an awesome interview"*. The model reads surface affect; the annotators recorded speaker stance, treating formulaic well-wishing as neutral. That is an annotation-convention disagreement rather than a model failure, and roughly a third of the confident error budget is probably unrecoverable without relabelling. The remaining three are the harder cases — polite surface with hostile intent (*"Take care bibi! Allah khair! No more stupidity will be tolerated now!"*, gold negative, predicted positive at $p = 0.95$) — which is genuine pragmatic inversion and exactly what a larger model would do better.

Language ID. Across 93,523 scored tokens, accuracy is 0.9699 overall, **0.9824 on the 68,085 tokens whose word appears as both Hindi and English in training**, 0.9363 on unambiguous words, and **0.7772 on the 3,569 tokens whose word was never seen in training**.

The first two numbers invert the expected ordering, and the label is misleading: a word gets flagged "ambiguous" by appearing with both tags, which in practice selects the highest-frequency function words — precisely those the model has the most evidence for. "Unambiguous" is dominated by the long tail seen once or twice. So that pair measures frequency, not context use, and we would drop the framing in a revision.

The unseen-word figure is the real generalisation result. With no memorised label available, the model reaches 0.777 working only from subword shape, skeleton bucket and surrounding context — against a 0.707 majority-class baseline at the HI/EN ratio of this corpus. The margin is real but modest, and it is the number we would put under the most scrutiny.

8 · Latency and throughput

Timings include CUDA synchronisation and 20 warmup iterations on a T4; the serving graph is encoder plus sentiment head.

Device / precision	Batch	p50 (ms)	p99	Throughput (sent/s)
GPU fp32	1	2.997	4.092	334
GPU fp32	8	3.198	4.147	2,502
GPU fp32	32	10.267	10.669	3,117
GPU fp32	256	84.134	85.671	3,043
GPU fp16	1	3.067	3.816	326

Device / precision	Batch	p50 (ms)	p99	Throughput (sent/s)
GPU fp16	128	10.046	10.296	12,742
CPU fp32	1	12.400	15.915	81
CPU int8	1	9.617	13.074	104

The knee sits between batch 8 and batch 32. Going from 1 to 8 costs 0.2 ms of latency and buys 7.5× the throughput — effectively free, since the model is launch-overhead bound at this size. Past 32, fp32 throughput saturates around 3,100 sentences/s and latency scales linearly with batch, so batching further only adds queueing delay. A production router should sit at batch 8–16.

fp16 changes the picture at high batch: identical at batch 1, but 12,742 sentences/s at batch 128 against 3,041 for fp32, a 4.2× gain once the tensor cores are saturated. Sequence length barely matters — p50 runs 2.816 / 2.982 / 3.125 / 2.928 / 3.011 ms at lengths 16 / 32 / 48 / 64 / 96, flat within noise. Attention is quadratic in principle, but at this width the kernel-launch floor dominates entirely, so length bucketing would buy nothing here and the full 96-token window is free.

Int8 dynamic quantisation on CPU is the clean win: **F1 0.8718 → 0.8724** (+0.0007, i.e. no measurable cost) with p50 falling 12.40 → 9.62 ms, a 1.29× speedup. If GPUs aren't in the budget, int8 on commodity CPU is the correct deployment.

One prediction we got wrong. We expected preprocessing to be a meaningful share of end-to-end latency at this model size. It measures **0.0228 ms per sentence** — 0.76% of the 2.997 ms forward pass. The word-level featuriser cache makes normalisation, skeleton hashing and tokenisation essentially free, and optimisation effort belongs in the model path after all. **End-to-end p50 at batch 1 is 3.020 ms**, comfortably inside the single-digit-millisecond requirement, at 3.77% of the parameter ceiling and a peak of 12,742 sentences/s.

9 • Limitations

The headline number is not a leaderboard number. Our test set is half the labelled validation file, the MLM stage saw that text unlabelled, and 1,500 rows is a fifth of the official evaluation size. Any of the three alone would warrant caution; together they mean 0.872 and 0.750 are not on the same axis. A clean comparison needs an inductive re-run with val/test text excluded from pretraining.

Domain mismatch with the stated use case. SentiMix is political Twitter. The problem statement is about support queues. Our qualitative probes — *"bohot badhiya service thi 😊"*, *"item delivered bol rha h but mila hi nhi 😞"* — all return neutral at 0.66–0.97

confidence, including cases where the emoji should flip the label. The model scores 0.872 in-domain and generalises poorly to the register the PS actually describes. That gap is the most important negative finding here and it does not appear anywhere in the aggregate metrics.

Pretraining scale. 1.81M tokens is three orders of magnitude below HingBERT. MLM loss was still falling at epoch 14.

One seed. Everything is seed 42, so differences of a point or two between slices aren't distinguishable from noise. The 6.9-point emoji delta is large enough to survive this; the 3.7-point English/Hindi slice gap is not.

Unresolved experiments. Drift augmentation and the skeleton channel are both active, so §6's robustness curve can't be attributed to the architecture until the ablation runs. The surface-feature channel isn't masked during MLM. And we argue against IndicBERT and mBERT from first principles and measured fertility without having fine-tuned them on this split.

Scope. The digraph folds are hand-built for Hindi Romanisation; Bangla, Tamil and Urdu follow different conventions, so the "South Asia" framing is broader than what has been validated. The folds are also imperfect within Hindi — `chahiye` → `c|ahie` is not a good decomposition.

10 · Next steps

The channel ablations come first: `use_skel` and `use_feats` are already wired as flags, so retraining each variant and overlaying the three drift curves turns the robustness claim into a measurement. Setting `augment_drift_p = 0.0` and comparing resolves the joint-claim problem. Both are ~25 minutes of T4 time each, which is the cheapest evidence available anywhere in this project.

Second, re-run inductively — exclude validation and test text from the MLM corpus — and report the delta. If 0.872 holds, the transductive objection is answered; if it drops materially, that is worth knowing before anyone else finds it.

Third, the baseline table: fine-tune IndicBERT, mBERT and HingRoBERTa on this split and report params, fertility, weighted F1 and batch-1 latency side by side. An 18.9M encoder within a few F1 of a 125M one, with the fertility column showing the fragmentation rather than asserting it, is the argument.

Fourth, address the domain gap. A few hundred labelled support-register utterances would show whether the neutral collapse in §9 is a fine-tuning-data problem or a representation problem. Beyond that, the largest accuracy lever is data: L3Cube-

HingCorpus is 52.93M sentences and 1.04B tokens, distributed via Google Drive rather than a raw URL — fetch it once, upload as a private dataset, point the corpus at it.

References

1. Patwa, P. et al. **SemEval-2020 Task 9: Overview of Sentiment Analysis of Code-Mixed Tweets**. ACL Anthology 2020. semeval-1.100 . CodaLab #20654.
2. Nayak, R., Joshi, R. **L3Cube-HingCorpus and HingBERT**. arXiv:2204.08398, 2022.
3. Devlin, J. et al. **BERT: Pre-training of Deep Bidirectional Transformers**. NAACL-HLT 2019.
4. Liu, Y. et al. **RoBERTa: A Robustly Optimized BERT Pretraining Approach**. arXiv:1907.11692, 2019.
5. Radford, A. et al. **Language Models are Unsupervised Multitask Learners**. 2019. (*byte-level BPE*)
6. Dao, T. et al. **FlashAttention**. NeurIPS 2022.

Reproducing this. One Kaggle notebook on a T4, internet on — 19.7 min pretraining, 4.9 min fine-tuning, ~30 min end to end including downloads and evaluation. `SMOKE_TEST = True` gives a ~3 minute verification pass; with internet off the loader scans `/kaggle/input` for the six source files. Seed 42; `d_model` 384, 6 layers, 6 heads, FFN 1,152; 16,000 BPE merges plus 1,291 emoji and placeholder tokens; 8,192 skeleton buckets; max length 96 subwords; MLM 14 epochs at batch 256 and lr 3e-4; fine-tuning 6 epochs at batch 64 and lr 1e-4; drift augmentation on 30% of rows. Outputs `best.pt` (75.5 MB), `pretrained.pt`, `tokenizer.json`, `config.json`, `metrics.json`, `results.png` and `inference.py`.